

Rule Name: Brute Force SSH Login Detection

Platform: SIEM / ELK Stack / Splunk

Rule Logic in Simple Words: If the same IP address fails to login more than 5 times within 60 seconds and then successfully logs in, trigger a critical alert and notify the SOC team immediately.

Rule Written in Splunk SPL Format:

```
index=auth_logs "Failed password" | stats count by src_ip, user | where count > 5 | eval threat="Brute Force Detected" | alert action=email notify="soc@company.com"
```

Rule Written in ELK Stack (Kibana) Format:

```
event.type: "authentication_failure" AND source.ip: * AND @timestamp: [now-60s TO now] AND count > 5
```

Rule Conditions:

Trigger condition: More than 5 failed logins from the same IP within 60 seconds followed by a successful login.

Severity level: Critical

Response action: Block IP on firewall, lock affected account, notify SOC team, create incident ticket.

2. Use Case: SQL Injection Attack

Use Case Name: SQL Injection Attack on Web Application Database

Background: A mid sized online shopping company runs a web application where customers can login, browse products, and make purchases. The web application connects to a backend database that stores customer names, email addresses, passwords, and credit card details.

What Happened: An attacker discovered that the login form of the web application was not properly secured. They started entering specially crafted SQL commands into the username field instead of a normal username. For example instead of typing a normal username the attacker typed:

```
' OR '1'='1' --
```

This SQL command tricks the database into thinking the login is valid and grants access without needing a real password. After getting in the attacker ran further SQL commands to extract the

entire customer database containing 50,000 customer records including names, emails, and hashed passwords.

What the SIEM Detected: The SIEM tool detected an unusually large number of database queries coming from a single IP address within a very short time. It also detected abnormal characters like single quotes and SQL keywords appearing in the web application's input fields which are classic signs of SQL injection.

Alert Generated: High severity alert triggered: SQL Injection attempt detected from IP 203.0.113.99 on the login page of the web application.

SOC Response: The L1 analyst received the alert and escalated to L2. The L2 analyst confirmed the SQL injection attack by analyzing the web server logs and database query logs. They immediately blocked the attacker's IP, took the vulnerable login page offline temporarily, and escalated to L3. The L3 expert patched the SQL injection vulnerability by implementing input validation and parameterized queries, restored the login page, and initiated a full investigation to determine how much data was accessed.

3. MITRE ATT&CK Mapping for SQL Injection Attack

Stage 1: Reconnaissance The attacker first visited the company's website and manually tested different input fields to see if they were vulnerable to SQL injection. They entered special characters and observed the error messages returned by the application to understand the database structure.

MITRE Tactic: Reconnaissance MITRE Technique: Active Scanning — Vulnerability Scanning

Stage 2: Initial Access The attacker successfully exploited the SQL injection vulnerability in the login form to bypass authentication and gain unauthorized access to the web application without needing a valid username or password.

MITRE Tactic: Initial Access MITRE Technique: Exploit Public Facing Application (T1190)

Stage 3: Execution After gaining access the attacker executed malicious SQL commands directly against the backend database to extract data. Commands like SELECT, UNION, and DROP were used to read, combine, and manipulate database tables.

MITRE Tactic: Execution MITRE Technique: Command and Scripting Interpreter (T1059)

Stage 4: Credential Access The attacker extracted the usernames and hashed passwords of all 50,000 customers from the database. They planned to use offline password cracking tools to convert the hashed passwords into plain text passwords.

MITRE Tactic: Credential Access MITRE Technique: Steal Web Session Cookie / Credentials from Database (T1555)

Stage 5: Collection The attacker collected all the sensitive customer data including names, email addresses, phone numbers, and credit card details from multiple database tables and organized it into a single file ready for transfer.

MITRE Tactic: Collection MITRE Technique: Data from Local System (T1005)

Stage 6: Defense Evasion To avoid being detected the attacker used encoding techniques to disguise their SQL commands so that basic web application firewalls could not recognize them as malicious input. They also deleted their activity from the web server access logs.

MITRE Tactic: Defense Evasion MITRE Technique: Obfuscated Files or Information (T1027)

Stage 7: Exfiltration Finally the attacker transferred the collected customer database file to their own remote server using an encrypted HTTPS connection to avoid triggering network monitoring alerts.

MITRE Tactic: Exfiltration MITRE Technique: Exfiltration Over Web Service (T1567)